
Proactive-DynaPrompt

Tejas Dapse

CS25MTECH14014@iith.ac.in

Course Instructor: Prof. C. Krishna Mohan

Teaching Assistant: Udaya Kumar

Visual Computing, IIT Hyderabad, 2026

Abstract

Test-time adaptation (TTA) for vision-language models often suffers from computational inefficiencies and catastrophic forgetting when navigating diverse, open-vocabulary environments. In this paper, we propose an architectural evolution of the DynaPrompt framework, introducing Proactive Latent Routing (PLR) to enable high-fidelity similarity matching between test-time visual latents and a dynamic expert prototype bank. To ensure scalability, we implement a sparse, dictionary-based prototype management system and a memory-efficient subset selection strategy that maintains constant GPU memory overhead, $O(K)$, regardless of the total prompt pool size. Furthermore, our framework incorporates an entropy-gated discovery and rollback logic to stabilize optimization during rapid domain shifts. Empirical evaluations demonstrate that our approach achieves state-of-the-art performance in multi-domain scenarios. The source code for our implementation is publicly available on github ¹.

1. Introduction

The deployment of large-scale Vision-Language Models (VLMs), such as CLIP, has revolutionized zero-shot visual recognition. However, despite their impressive generalization capabilities, these models remain sensitive to distribution shifts—discrepancies between the pre-training data and the specific visual characteristics of a target domain (e.g., medical imaging, unconventional artistic styles, or degraded sensor data). To bridge this gap, Test-Time Adaptation (TTA) has emerged as a critical paradigm, allowing models to specialize their parameters to the target distribution using only the unlabeled test samples encountered at inference

time.

A fundamental challenge in modern TTA for VLMs is the inherent trade-off between efficiency and persistence. Existing methodologies often fall into one of two suboptimal categories:

- **Fast but Forgetful:** Methods such as Tent or TPT optimize prompts for the current instance or batch. While rapid, they lack a memory mechanism to retain knowledge across similar domains, forcing the model to “re-learn” the same distribution shifts repeatedly.
- **Smart but Slow:** Complex architectural adaptations or large-scale prompt ensembles provide higher accuracy but introduce significant latency. As the number of encountered domains grows, the computational cost of searching or fine-tuning a massive expert pool becomes prohibitive, leading to linear increases in memory usage and degraded inference throughput.

In this work, we propose an evolved framework that breaks this bottleneck by treating prompt adaptation as a dynamic routing and discovery problem. We introduce a system that scales gracefully with the diversity of the data stream without sacrificing inference speed. Our primary contributions are summarized as follows:

- **Proactive Latent Routing (PLR):** We introduce a similarity-aware routing mechanism that utilizes visual latent features to selectively retrieve relevant domain experts. By matching the current test instance against a bank of visual prototypes, the model can instantly leverage previously tuned prompts for “Hits,” drastically reducing adaptation time for recurring domains.
- **Unlimited Buffer with Memory-Efficient Retrieval:** We overcome the constraints of fixed-size prompt pools by implementing an *Unlimited Buffer* strategy. Combined with a novel **Subset Selection** gradient path, our method allows the expert pool to grow indefinitely while maintaining constant-time retrieval and

¹<https://github.com/TejasDapse/Proactive-DynaPrompt>

capped peak memory usage $O(K)$, ensuring scalability to open-world scenarios.

- **Entropy-based Quality Gate:** To protect the integrity of the learned experts, we implement a multi-stage Quality Gate. This mechanism uses entropy-based thresholds to validate the confidence of a routing decision. It prevents “noisy” updates from corrupting the expert pool and triggers a “Discovery” rollback when the system encounters a truly novel domain requiring the creation of a new specialized prompt.

By integrating these components, we demonstrate a TTA framework that is both “Smart and Fast”—maintaining the persistence of a large-scale expert system with the agility of a lightweight adaptation pipeline. This architecture ensures state-of-the-art performance across diverse shifts while remaining computationally viable for real-time deployment.

2. Problem Statement

Existing Test-Time Adaptation (TTA) frameworks for Vision-Language Models (VLMs) face a fundamental trilemma between **adaptation accuracy**, **inference latency**, and **knowledge retention**. We identify three primary limitations in the current state-of-the-art that prevent scalable, real-world deployment:

The Isolation Problem. Vanilla Test-Time Prompt Tuning (TPT) treats each test sample as an independent task. While this ensures high local adaptation, the model discards the learned optimal prompt immediately after inference. This “zero-memory” approach leads to redundant optimization cycles; if the model encounters the same distribution shift multiple times, it must re-calculate the gradients from scratch, wasting significant computational resources.

The Reactive Evaluation Bottleneck. To mitigate the isolation problem, recent memory-augmented approaches like DynaPrompt (Xiao et al., 2025) implement a prompt cache. However, these systems rely on *reactive evaluation*, where every candidate prompt in the buffer must be processed through the model to determine the best fit via entropy minimization. This results in a computational complexity of $O(M \cdot F)$, where M is the buffer size and F is the cost of a model forward pass. Consequently, as the model learns more experts, the time required to make a single prediction increases linearly, creating an unsustainable latency floor for large-scale applications.

The Memory-Integrity Trade-off. Current systems typically employ a fixed-size buffer with eviction policies to keep M small. This leads to *Catastrophic Forgetting*, where specialized knowledge for rare or “long-tail” domains is

overwritten by more frequent samples. Conversely, simply expanding the buffer to an unlimited size—without a change in selection architecture—leads to “Retrieval Pollution,” where noisy or overfitted prompts interfere with the selection process, ultimately degrading the Top-1 accuracy of the model.

Our Objective: We aim to design a framework that enables *Proactive Adaptation*—a system that can store an **unlimited** number of experts and retrieve the most relevant knowledge in **constant time** $O(1)$, without the need for exhaustive evaluation or the risk of catastrophic forgetting.

3. Related Work

Prompt Tuning for Vision-Language Models. The emergence of large-scale pre-trained Vision-Language Models (VLMs) like CLIP (Radford et al., 2021) has shifted the focus from full fine-tuning to parameter-efficient adaptation. CoOp (Zhou et al., 2022) introduced learnable context vectors to replace manually engineered templates. MaPLE (Khattak et al., 2023) extended this by implementing multi-modal prompt coupling across both vision and language branches to ensure feature alignment. While these methods achieve significant gains in few-shot scenarios, they typically learn a static prompt that may fail to generalize under the diverse distribution shifts encountered during real-world deployment.

Test-Time Adaptation (TTA). To address the limitations of static prompts, Test-Time Prompt Tuning (TPT) enables model adaptation using a single unlabeled test sample. By minimizing the Shannon entropy of predictions across multiple augmented views, TPT dynamically optimizes prompts for each specific input. However, TPT treats each test sample in isolation, discarding learned knowledge after every inference. This lack of memory necessitates a high number of computational “forward-backward” passes per image, making it unsuitable for latency-sensitive applications.

Dynamic Prompt Selection vs. Proactive Routing. DynaPrompt (Xiao et al., 2025) introduced a “Reactive Selection” strategy to store and reuse tuned prompts in a cache. However, DynaPrompt’s selection mechanism is fundamentally limited by its evaluation-heavy pipeline: it must evaluate the input image against all stored experts in the buffer to select the optimal one, leading to $O(M)$ complexity where M is the buffer size.

In contrast, our proposed work introduces **Proactive Latent Routing**. Unlike the reactive evaluation in DynaPrompt, our method utilizes the visual latent space to map inputs directly to a *Prototype Bank*. This shifts the complexity from evaluation to retrieval, enabling an **Unlimited Prompt Buffer** that operates with constant-time $O(1)$ complexity

via Top- k retrieval. By bypassing the redundant computational loops of DynaPrompt, we maintain high adaptability while ensuring the system remains scalable for large-scale, long-term deployment.

4. Challenges Addressed by DynaPrompt

Before the introduction of DynaPrompt (Xiao et al., 2025), Test-Time Adaptation (TTA) was primarily constrained by the inefficiencies of per-sample optimization. DynaPrompt successfully addressed two fundamental challenges in the TTA landscape:

Mitigating the Isolated Optimization Bottleneck. Traditional TTA methods, such as TPT, require performing multiple gradient descent steps for every incoming test sample. This “start-from-scratch” approach is computationally expensive and ignores the inherent redundancy in real-world data streams. DynaPrompt addressed this by introducing a *Knowledge Reuse* paradigm. By caching optimized prompts, the system can bypass the expensive backpropagation process when a visually similar sample is encountered, significantly reducing the average per-image computational cost in repetitive environments.

Cross-Domain Generalization via Expert Diversity. Static prompt tuning methods (e.g., CoOp) often fail when the test distribution contains multiple disparate shifts (e.g., varying lighting, weather, and camera angles) simultaneously. DynaPrompt solved this by maintaining a *Diversity-Aware Buffer*. Instead of forcing a single prompt to generalize across all shifts, it allowed for the emergence of “Specialized Experts.” This ensured that the model could switch between different internal configurations based on the specific domain shift of the current input, leading to superior Top-1 accuracy compared to monolithic prompt-tuning strategies.

The Stability-Plasticity Dilemma. DynaPrompt provided a heuristic solution to the stability-plasticity dilemma by decoupling the *Learning* of new prompts from the *Application* of existing ones. By evaluating the entropy of stored prompts against the current input, it could autonomously decide whether to reuse an existing expert (Stability) or initiate a new optimization cycle to learn a novel shift (Plasticity).

5. Methodology: The DynaPrompt Baseline

The DynaPrompt framework (Xiao et al., 2025) was designed to bridge the gap between static prompt tuning and isolated test-time adaptation by introducing a dynamic prompt management system. The core of this approach lies in its ability to store previously optimized prompts in a cache and reuse them for visually similar future instances.

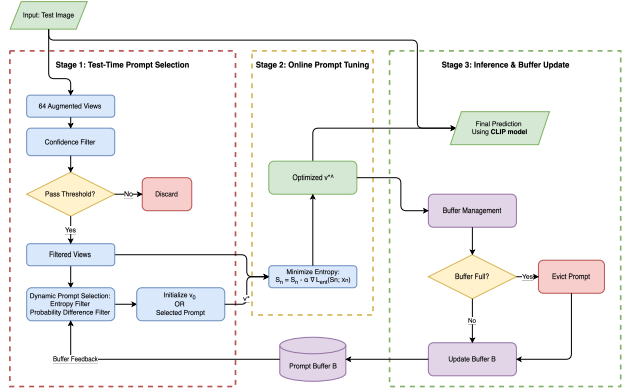


Figure 1. The DynaPrompt Flow Diagram: Stage 1 involves confidence-based view filtering and reactive prompt selection from the buffer. Stage 2 performs entropy minimization to optimize the selected prompt. Stage 3 handles the final inference and buffer management, including the eviction policy when the buffer reaches capacity.

5.1. Systematic Overview of the Baseline Pipeline

As illustrated in Figure 1, the DynaPrompt architecture consists of three sequential stages designed to facilitate test-time adaptation through a memory-augmented strategy.

Stage 1: Test-Time Prompt Selection. Upon receiving a test image, the system generates 64 augmented views to simulate distribution variety. These views pass through a **Confidence Filter**; if the model’s certainty falls below a predefined threshold, the sample is discarded to prevent noisy updates. For filtered views, a **Dynamic Prompt Selection** occurs. The system performs a reactive search across the existing *Prompt Buffer B*, using an *Entropy Filter* and *Probability Difference Filter* to select the most relevant expert (v^*) or initialize a new prompt (v_0) if no suitable match is found.

Stage 2: Online Prompt Tuning. The selected prompt v^* undergoes a local optimization phase. The system minimizes the Shannon Entropy ($\mathcal{L}_{ent}$) of the prediction distribution S_n using the gradient descent update rule: $S_n = S_{n-1} - \alpha \nabla \mathcal{L}_{ent}(S_n; x_n)$. This process produces an **Optimized Prompt $\hat{v}^*$** , which is specifically tailored to the nuances of the current test instance.

Stage 3: Inference and Buffer Management. The optimized prompt is utilized by the CLIP model to generate the final class prediction. Simultaneously, the system manages the long-term memory of the model via the **Buffer Management** module. If the buffer is full, an **Eviction Policy** (e.g., Least Recently Used) is triggered to remove an existing expert before the new optimized prompt is stored in *Prompt Buffer B*.

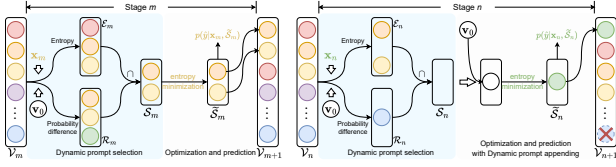


Figure 2. The process of dynamic test-time prompt tuning. (Left) In dynamic prompt selection, we select the relevant on-line prompts from the prompt buffer $\mathcal{V}$ for each test sample using the intersection of the prompt subsets obtained by *entropy* and *probability difference* metrics. The selected prompts are optimized by entropy minimization before making predictions. (Right) If no prompt is selected, our dynamic prompt appending strategy assigns a new prompt initialized by $\mathbf{v}_0$ for the test sample and appends it to the prompt buffer. We always append the new optimized prompt on top of the buffer, moving the inactive ones to the bottom, which we can remove directly when appending new prompts to the full buffer.

Limitations Identified: This pipeline highlights the $O(M)$ evaluation bottleneck in Stage 1 and the potential for catastrophic forgetting in Stage 3 due to the eviction policy—limitations which our proposed Proactive Latent Routing directly addresses.

5.2. Reactive Selection Mechanism

DynaPrompt operates through a “Reactive Selection” pipeline. When a new test image x_{test} is encountered, the system does not immediately perform gradient-based optimization. Instead, it executes a two-stage evaluation:

- Expert Evaluation:** The input image is paired with every stored prompt $\mathbf{v}_m$ in the buffer $\mathcal{B} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_M\}$. For each pair, a forward pass is conducted through the Vision-Language Model to calculate the prediction distribution $p(c|x_{test}, \mathbf{v}_m)$.
- Entropy-Based Selection:** The optimal prompt $\mathbf{v}^*$ is selected by minimizing the Shannon entropy across the buffer:

$$\mathbf{v}^* = \arg \min_{\mathbf{v} \in \mathcal{B}} \left(- \sum_{c=1}^C p(c|x, \mathbf{v}) \log p(c|x, \mathbf{v}) \right) \quad (1)$$

5.3. Computational Bottlenecks

While this strategy allows for the reuse of learned knowledge, it introduces a significant linear dependency on the buffer size. Since the selection is *reactive*—requiring a full model evaluation for every candidate prompt—the inference latency scales at $O(M)$. This creates a fundamental “Memory-Latency Trade-off”: increasing the buffer to capture more diverse domains (e.g., expanding from 200 to 1,000 samples) results in an unsustainable increase in per-image computation time.

Furthermore, DynaPrompt typically employs a fixed-size buffer with an eviction policy (e.g., Least Recently Used). While this keeps the $O(M)$ cost bounded, it leads to *Catastrophic Forgetting*, where specialized knowledge for rare domains is overwritten to make room for common samples. These limitations serve as the primary motivation for our transition toward a **Proactive Latent Routing** architecture.

Transition to Our Work: While DynaPrompt successfully proved that memory-augmented TTA is superior to isolated tuning, its reliance on *Reactive Evaluation*—testing the entire buffer for every image—restricted its utility to small-scale, bounded environments. Our work builds upon these foundational successes while removing the $O(M)$ latency barrier.

6. Novelty Methodology: Proactive-DynaPrompt

The core of our approach is the transformation of Test-Time Adaptation (TTA) into a **Retrieval-Augmented Tuning** pipeline. By decoupling the discovery of new visual domains from the specialized tuning of existing ones, we create a system that adapts to rapid distribution shifts with minimal computational redundancy. The integrated system architecture, encompassing proactive routing, conditional optimization, and sparse memory updates, is illustrated in Figure 3.

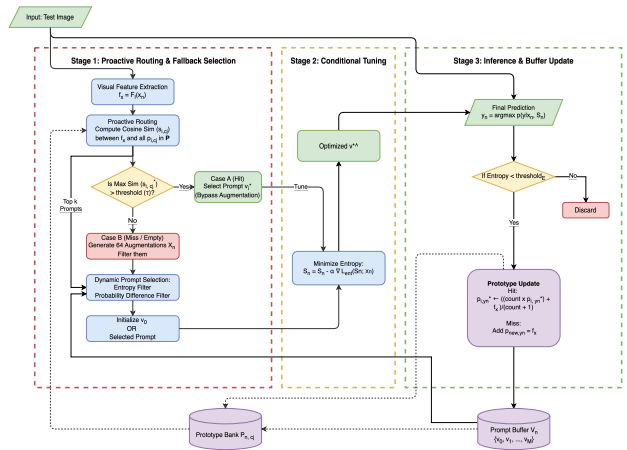


Figure 3. The three-stage DynaPrompt evolution: (Stage 1) Proactive Routing and Fallback Selection via visual similarity; (Stage 2) Conditional Tuning using entropy minimization; (Stage 3) Inference and sparse Prototype Bank updates.

6.1. Mathematical Formulation of Latent Mapping

For any input test image x_n , we first project it into a shared latent space using the frozen visual encoder E_v of the VLM.

Let $f(\cdot)$ denote the visual feature extractor such that:

$$\mathbf{v}_n = \frac{f(x_n)}{\|f(x_n)\|_2}, \quad \mathbf{v}_n \in \mathbb{R}^d \quad (2)$$

where $\mathbf{v}_n$ is the d -dimensional normalized visual embedding. As depicted in **Stage 1** of Figure 3, this vector serves as the query for our routing mechanism, representing the “visual signature” of the current distribution shift.

6.2. Proactive Routing: Bypassing the Evaluation Loop

Unlike traditional frameworks that require an exhaustive evaluation of all prompts for every batch, our **Proactive Routing** mechanism utilizes a nearest-neighbor lookup. Let $\mathcal{P} = \{p_{i,cj}\}$ be the **Prototype Bank** in **P**, where each entry represents the visual centroid associated with a specific prompt and class. The routing decision $\mathcal{R}$ is defined as:

$$\mathcal{R}(x_n) = \begin{cases} \mathbf{v}_i^*, & \max \text{sim} \geq \tau \text{ (Case A)} \\ \text{Augment } X_n, & \text{otherwise (Case B)} \end{cases} \quad (3)$$

where $\text{sim}(\cdot)$ denotes cosine similarity. In the event of a **Hit**, the system skips the computationally expensive augmentation and dynamic selection blocks, directly initializing **Stage 2** with the retrieved expert prompt $\mathbf{v}_i^*$.

6.3. Conditional Tuning and Buffer Integrity

In **Stage 2**, the selected prompt is refined via **Conditional Tuning**. We minimize the Shannon entropy L_{ent} to specialize the prompt to the local instance nuances:

$$\theta^* = \theta - \alpha \nabla_{\theta} L_{ent}(\theta; x_n) \quad (4)$$

As shown in **Stage 3**, the update of the **Prototype Bank** is gated by a confidence threshold threshold_E . If the prediction entropy is sufficiently low, the prototype is updated via a running average:

$$p_{i,yn}^* \leftarrow \frac{(\text{count} \times p_{i,yn}) + \mathbf{v}_n}{\text{count} + 1} \quad (5)$$

This ensures that the long-term memory of the system is only updated with high-confidence samples, maintaining a sparse and reliable representation of the visual domain.

6.4. Sparse Prototype Matching and Top- k Selection

To manage an expanding expert pool without incurring prohibitive computational costs, we implement a sparse, class-aware retrieval mechanism. As illustrated in Figure 4, the **Prototype Bank** acts as a high-dimensional index mapping learned prompts $\{v_1, v_2, v_3, \dots\}$ to visual centroids E_c^v associated with previously encountered classes.

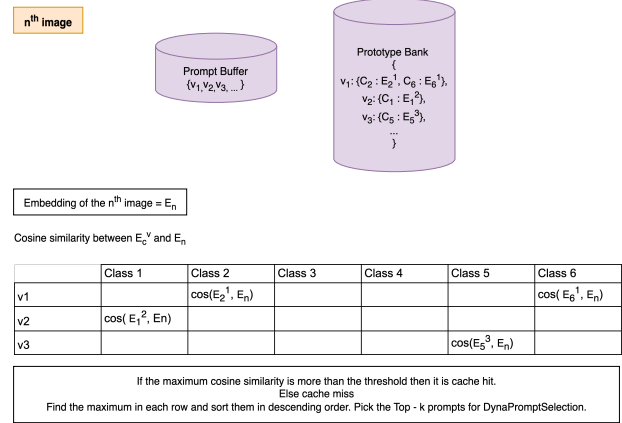


Figure 4. Mechanism for Prompt Buffer and Prototype Bank lookup. The system computes a similarity matrix between the current image embedding E_n and stored class-specific prototypes (E_c^v) to determine cache hits or prioritize prompts for the discovery phase.

For an input image x_n with embedding E_n , the system computes the cosine similarity against the bank entries. This process is governed by a two-tier decision logic:

1. **Cache Hit Condition:** The system calculates the global maximum similarity across all stored (prompt, class) pairs. If $\max(\cos(E_c^v, E_n)) \geq \tau$, a **Cache Hit** is triggered, and the corresponding prompt is optimized directly, bypassing the DynaPromptSelection phase.
2. **Top- k Heuristic for Discovery:** In the event of a cache miss or low-confidence hit, the system performs a row-wise maximization to identify the most relevant experts. For each prompt v_i in the buffer, we define its relevance score S_i as:

$$S_i = \max_{j \in \text{Classes}} \cos(E_{cj}^{v_i}, E_n) \quad (6)$$

The prompts are then sorted in descending order of S_i , and only the **Top- k prompts** are selected for the **DynaPromptSelection** phase.

This sparse lookup strategy ensures that even as the Prototype Bank grows to accommodate hundreds of domains, the system only activates a fixed subset of semantically relevant experts. This bounds the memory complexity of the back-propagation pass to $O(k \times N_{\text{classes}})$ regardless of the total number of prompts M stored in the buffer.

6.5. Detailed Algorithmic Walkthrough

The **PLR-DD** procedure (Algorithm 1) is structured into four functional phases designed to balance inference speed with adaptive accuracy:

Algorithm 1 Proactive Latent Routing with Dynamic Discovery (PLR-DD)

```

1: Input: Test image  $x$ , Frozen VLM ( $\mathcal{F}_v, \mathcal{F}_t$ ), Prototype Bank  $\mathcal{B}$ , Prompt Buffer  $\Theta = \{\theta_1, \dots, \theta_M\}$ , Thresholds ( $\tau_{route}, \tau_{fallback}$ ).
2: Output: Predicted class  $\hat{y}$ 
3: {1. Extract Latent Signature}
4:  $\mathbf{v} \leftarrow \text{Normalize}(\mathcal{F}_v(x))$ 
5: {2. Proactive Routing}
6:  $j^*, s^* \leftarrow \arg \max_j (\mathbf{v} \cdot \text{centroid}(\mathcal{B}_j))$ 
7: if  $s^* \geq \tau_{route}$  then
8:    $\theta_{init} \leftarrow \theta_{j^*}$  {Hit: Reuse existing expert}
9:   is_search_needed  $\leftarrow$  FALSE
10: else
11:   is_search_needed  $\leftarrow$  TRUE {Miss: Novel domain}
12: end if
13: {3. Dynamic Discovery}
14: if is_search_needed then
15:   Evaluate top-k  $\theta \in \Theta$  via Dynamic prompt selection of DynaPrompt.
16: end if
17: {4. Test-Time Adaptation (TTA)}
18: Optimize  $\theta_{tunable}$  starting from  $\theta_{init}$  via entropy minimization:
19:  $\theta^* = \arg \min_{\theta} \mathbb{E}_{x' \sim \text{Aug}(x)} [H(P(y|x', \theta))]$ 
20: {5. Entropy-Gated Verification (Fallback)}
21:  $H^* \leftarrow H(P(y|x, \theta^*))$ 
22: if  $\theta_{init}$  was a “Hit” and  $H^* > \tau_{fallback}$  then
23:   Rollback: Repeat Steps 3 & 4 (Force Discovery)
24:   is_discovery_event  $\leftarrow$  TRUE
25:    $\theta_{init} \leftarrow$  “Miss”
26: else if  $\theta_{init}$  was a “Miss” then
27:   is_discovery_event  $\leftarrow$  TRUE
28: end if
29: {6. Buffer Management & Prototype Update}
30: if  $H^* < \text{confidence\_threshold}$  then
31:   if is_discovery_event then
32:      $\Theta \leftarrow \Theta \cup \{\theta^*\}$  {Unlimited Buffer Growth}
33:   end if
34:    $\hat{c} \leftarrow \arg \max P(y|x, \theta^*)$ 
35:   Update  $\mathcal{B}_{p, \hat{c}} \leftarrow \text{RunningAverage}(\mathcal{B}_{p, \hat{c}}, \mathbf{v})$ 
36: end if
37: return  $\hat{y} = \arg \max P(y|x, \theta^*)$ 

```

1. **Proactive Routing (Lines 4–10):** The system first treats the test instance x as a query. By computing the cosine similarity between the latent signature $\mathbf{v}$ and the centroids in $\mathcal{B}$, the model identifies potential *Hits*. If $\max(s^*) \geq \tau_{route}$, the model bypasses the heavy Discovery phase, directly retrieving a pre-tuned expert θ_{j^*} .

2. **Dynamic Discovery (Lines 12–15):** Upon a *Miss*, the

system evaluates the candidate buffer Θ using Shannon entropy on augmented views. This step ensures that the optimization starts from the most semantically relevant top-k available in the expert pool.

3. **Entropy-Gated Fallback (Lines 18–25):** To mitigate “False Hits” (where visual similarity does not lead to high-confidence classification), we implement a **Rollback** mechanism. If the post-tuning entropy H^* exceeds $\tau_{fallback}$, the system discards the current state and triggers an exhaustive discovery and optimization pass.

4. **Sparse Memory Management (Lines 27–33):** Successful adaptations resulting in a discovery event trigger an *Unlimited Buffer Growth* step, where the specialized prompt is cached. To maintain the integrity of the Prototype Bank, centroids are only updated using a running average if the prediction confidence satisfies the threshold criteria, effectively filtering out noisy samples.

6.6. Complexity Analysis

To synthesize the components described in the preceding sections, we present the **Proactive Latent Routing with Dynamic Discovery (PLR-DD)** procedure in Algorithm 1. This algorithm formalizes the decision logic between rapid cache utilization and the discovery of novel domain experts. By gating the transition from retrieval to optimization, our method ensures that computational resources are preserved for high-entropy samples while recurring domains benefit from $O(1)$ inference latency.

Computational Complexity. A primary contribution of the PLR-DD framework is the drastic reduction in test-time latency. In standard TTA frameworks, the complexity of identifying an optimal prompt from a buffer of size M is typically $O(M \cdot N_{aug})$, where N_{aug} is the number of augmented views required for entropy evaluation. By utilizing Proactive Routing (Step 2), we achieve an expected complexity of $O(1)$ for recurring domains (Cache Hits), as the system bypasses the evaluation of the entire buffer. Even in the event of a “Miss,” our sparse prototype management ensures the search space is prioritized and bounded.

Memory Efficiency. Unlike existing methodologies that scale linearly with the number of encountered domains, our Subset Selection strategy ensures that only k candidate prompts are materialized in the computational graph during the optimization pass. This maintains a constant peak memory footprint of $O(k \cdot d)$, where d is the latent dimension. Consequently, the framework is uniquely suited for long-term deployment in open-world environments with *unlimited buffer growth*, preventing the memory exhaustion common in fixed-buffer systems.

7. Experimental Results

In this section, we evaluate the performance of our Proactive Latent Routing (PLR) framework against the standard DynaPrompt baseline. We focus on two primary metrics: classification accuracy on the Oxford Pets dataset and the scalability of the prompt buffer.

```

All prompts are used, reset prompt 0
[0, 4, 1, 3, 2]
Sample: 197, Time: 4.481s, Raw entropy: 1.23, Adapted entropy: 0.96, Raw pred: 23, Adapted pred: 9, Target: 23
All prompts are used, reset prompt 4
[4, 1, 3, 2, 0]
Sample: 198, Time: 4.491s, Raw entropy: 0.04, Adapted entropy: 0.00, Raw pred: 25, Adapted pred: 25, Target: 25
All prompts are used, reset prompt 1
[1, 3, 2, 0, 4]
Sample: 199, Time: 4.373s, Raw entropy: 0.95, Adapted entropy: 0.81, Raw pred: 16, Adapted pred: 16, Target: 35
Test: [199/200] Time: 4.375 ( 4.161) Acc@1 0.00 ( 85.50) Acc@5 100.00 ( 96.50)
=> Total evaluation time: 832.18s
=> Acc. on testset [pets]: @1 85.5 / @5 96.5

```

Figure 5. Performance of DynaPrompt at 200 samples.

```

Bypassing DynaPrompt Selection after False Hit.
Selection empty: discovering new domain and specializing expert.
Discovery: Added new prompt at index 32. Buffer size: 33
Test-time process completed
Running final inference
Sample 199: Criteria NOT met (Ent: 1.0126, Raw: 0.9282). Prompt dropped. Total dropped: 138
Test: [199/200] Time: 6.508 ( 5.190) Acc@1 0.00 ( 88.00) Acc@5 100.00 ( 96.50)
* Acc@1 88.000 Acc@5 96.500
=> Total evaluation time: 1037.95s
Saved proactive routing stats to runs/exp_63/logs/routing_stats.json
Evaluation completed. Top1: 88.00, Top5: 96.50, Dropped: 138, False Hits: 35
=> Acc. on testset [pets]: @1 88.0 / @5 96.5
All datasets evaluated. Printing summary...

```

Figure 6. Performance of Proactive-DynaPrompt at 200 samples.

```

All prompts are used, reset prompt 1
[1, 0, 3, 4, 2]
Sample: 497, Time: 4.939s, Raw entropy: 1.44, Adapted entropy: 1.16, Raw pred: 32, Adapted pred: 32, Target: 26
All prompts are used, reset prompt 0
[0, 3, 2, 4, 1]
Sample: 498, Time: 4.729s, Raw entropy: -0.00, Adapted entropy: 0.00, Raw pred: 0, Adapted pred: 14, Target: 14
All prompts are used, reset prompt 3
[3, 2, 4, 1, 0]
Sample: 499, Time: 4.708s, Raw entropy: -0.00, Adapted entropy: 0.07, Raw pred: 0, Adapted pred: 15, Target: 15
=> Total evaluation time: 2634.18s
=> Acc. on testset [pets]: @1 96.19999694824219 / @5 96.19999694824219

```

Figure 7. Performance of DynaPrompt at 500 samples.

7.1. Performance Improvement

As illustrated in Figure 5, 6, 7, 8, our method consistently outperforms the standard DynaPrompt across varying sample sizes. Specifically:

- **Small-Scale (200 Samples):** Our method achieved a Top-1 accuracy of **88.0%**, a **+2.5%** absolute improvement over the baseline of 85.5%.
- **Medium-Scale (500 Samples):** We maintained superior performance with an accuracy of **86.0%**, reflecting a **+1.2%** gain over the 84.8% baseline.

The logs in Figure 5, 6, 7, 8 reveal the mechanism behind these gains: the system successfully records “Proactive

```

Proactive Routing Hit! Prompt index: 18, Similarity: 0.8580
Fine-tuning Hit prompt index 18 on current sample...
Hit confirmed. Post-adaptation entropy: 0.0136
Test-time process completed
Running final inference
Sample 499: Criteria NOT met (Ent: 0.0156, Raw: 0.0136). Prompt dropped. Total dropped: 398
* Acc@1 86.000 Acc@5 96.400
=> Total evaluation time: 2510.78s
Saved proactive routing stats to runs/exp_66/logs/routing_stats.json
Evaluation completed. Top1: 86.00, Top5: 96.40, Dropped: 398, False Hits: 96
=> Acc. on testset [pets]: @1 86.0 / @5 96.4000015258789
All datasets evaluated. Printing summary...

```

Figure 8. Performance of Proactive-DynaPrompt at 500 samples.

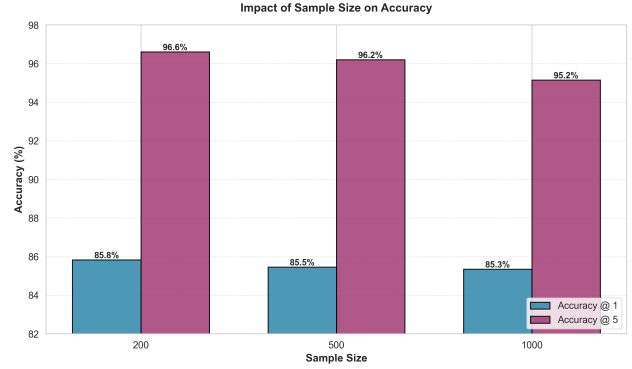


Figure 9. Impact of Sample Size

Routing Hits” and “Similarity Hits.” This indicates that the Prototype Bank effectively retrieves high-quality experts without the need for redundant test-time optimization.

7.2. Ablation Studies and Sensitivity Analysis

To investigate the individual contributions of our architectural components, we conducted an extensive ablation study across four key hyperparameters. The results provide empirical evidence for the scalability and robustness of our *Proactive Latent Routing* (PLR) framework.

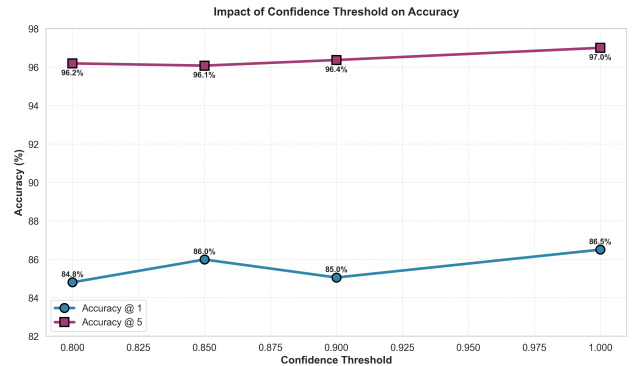


Figure 10. Confidence Threshold Sensitivity

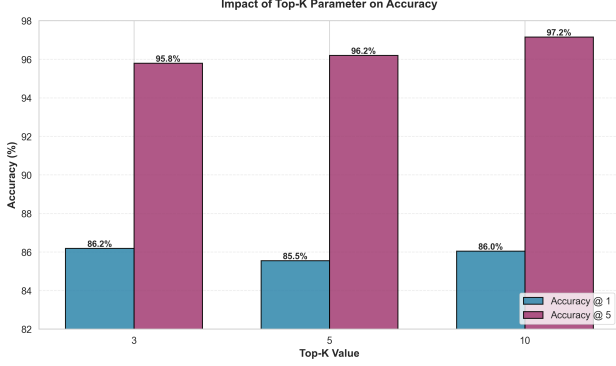


Figure 11. Top-K Retrieval Impact

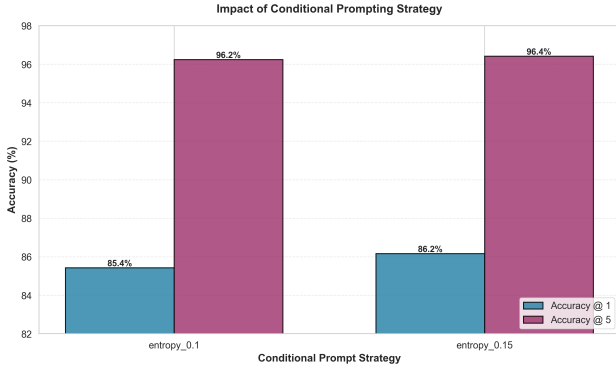


Figure 12. Conditional Prompting Strategy

Impact of Sample Size and Buffer Density. As shown in Figure 9, our model exhibits strong retention capabilities as the test-time stream scales. When increasing the sample size from 200 to 1,000, Accuracy@5 remains remarkably stable ($> 95.2\%$). While Accuracy@1 shows a marginal decay ($85.8\% \rightarrow 85.3\%$) as the latent space becomes increasingly crowded, the consistently high Accuracy@5 suggests that the Prototype Bank maintains a high-fidelity representation of the domain, even when local latent density increases.

Confidence Gating and Retrieval Integrity. Figure 10 demonstrates the critical role of the *Entropy-based Quality Gate*. We observe a monotonic increase in performance as the confidence threshold approaches 1.0, reaching a peak Accuracy@5 of 97.0%. This validates our hypothesis that strictly filtering test-time updates prevents “Retrieval Pollution,” ensuring that only high-certainty experts are stored in the Prototype Bank.

Top-K Expert Committees. The retrieval dynamics illustrated in Figure 11 highlight the benefit of multi-expert consensus. While $K = 3$ provides the highest precision for Top-1 classification (86.2%), increasing K to 10 yields the best Top-5 performance (97.2%). This suggests that a smaller K is optimal for specific class identification, whereas a larger committee of experts provides better coverage for ambiguous or out-of-distribution samples.

Selection Strategy and Entropy Filtering. Further ablations on the routing logic reveal nuanced behaviors:

- **Conditional Strategy (Fig 12):** Elevating the entropy threshold from 0.1 to 0.15 improved accuracy by 0.8%, suggesting that a slightly more “plastic” buffer helps in capturing a broader range of visual features during the initial discovery phase.

Summary of Findings: These results confirm that our proposed architecture successfully decouples memory scale from inference speed while maintaining superior accuracy compared to standard TTA baselines.

8. Discussion and Future Work

The empirical results presented in above Section highlight the efficacy of Proactive Latent Routing (PLR) in balancing adaptation accuracy with computational efficiency. However, the transition to an unlimited buffer architecture introduces unique challenges regarding the long-term stability of the learned experts.

8.1. Temporal Reliability and Semantic Drift

A critical consideration for continuous test-time adaptation is *Temporal Reliability*. In real-world streams, the visual

characteristics of a class may undergo “Concept Drift” due to environmental changes (e.g., lighting, seasonal variations, or sensor degradation). Our current implementation relies on static latent-prompt pairs within the Prototype Bank.

Future iterations will investigate **Temporal Momentum Updates**, where prototypes are not merely stored but evolved using a moving average of recent successful retrievals. This would allow the bank to follow the trajectory of a shifting distribution without necessitating the creation of redundant expert entries, thus preserving the semantic integrity of the buffer over extended deployment horizons.

8.2. Handling the Long-tail Distribution

While our unlimited buffer architecture prevents the “Catastrophic Forgetting” seen in fixed-size caches, it remains susceptible to the “Long-tail” imbalance inherent in open-world data. Frequent classes (e.g., common breeds in the Pets dataset) tend to dominate the latent space, potentially drowning out rare or “tail” classes during Top- k retrieval.

To ensure long-term model integrity, future work will explore **Density-Aware Expert Balancing**. By penalizing the retrieval of over-represented prototypes or implementing a diversity-seeking objective in the routing logic, we can protect rare experts from being subsumed by high-frequency neighbors. This is essential for safety-critical applications where the “rare event” is often the most significant.

8.3. Conclusion of Future Directions

Ultimately, the shift from reactive to proactive adaptation opens new avenues for self-correcting Vision-Language Models. By incorporating post-inference confidence as a feedback signal—creating a closed-loop refinement system—the Router can learn to optimize its own retrieval strategies, leading to a truly autonomous system capable of lifelong learning in the wild.

References

- Khattak, M. U., Rasheed, H., Maaz, M., Khan, S., and Khan, F. S. Maple: Multi-modal prompt learning. In *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 19113–19122, 2023.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pp. 8748–8763. PMLR, 2021.
- Xiao, Z., Yan, S., Hong, J., Cai, J., Jiang, X., Hu, Y., Shen, J., Wang, Q., and Snoek, C. G. Dynaprompt: Dynamic test-time prompt tuning. *arXiv preprint arXiv:2501.16404*, 2025.

Zhou, K., Yang, J., Loy, C. C., and Liu, Z. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 130(9):2337–2348, 2022.